UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Componentes de Lógica de Negocios
Producto Integrador de Aprendizaje
Proyecto Final: Implementación de una solución de software basada en lógica de negocio
(25 puntos de la calificación total)

	D	L	M	M	J	V	S
S1	1	2	3	4	5	6	7
S2	8	9	10	11	12	13	14
S3	15	16	17	18	19	20	21
S4	22	23	24	25	26	27	28
S5	29	30					

Noviembre 2026
Fecha de entrega de documentos y repositorio:
06 Noviembre 2026
Fecha de presentación y defensa:
9 al 20 Noviembre 2026.
No se aceptarán proyectos después de la fecha de entrega establecida.

El proyecto deberá cumplir con todas las especificaciones solicitadas:
1.- Cada equipo continuará con la misma empresa/caso de negocio que le fue asignado y desarrollado durante las Unidades 2 y 3. No se cambia de empresa para el PIA.
2.- Integrar y refinar los productos elaborados previamente (BRD, User Stories, Gherkin, BPMN y glosario) y utilizarlos como base obligatoria de la arquitectura e implementación.
3.- Implementar una aplicación funcional que materialice las reglas y procesos del negocio, incorporando DDD, Specification, State Machine y una API documentada.
4.- Mantener el proyecto en un repositorio GitHub con la estructura mínima indicada en este documento, pruebas automatizadas y pipeline CI/CD activo.
5.- Contenerizar la solución con Docker Compose para DB + App + Tests, separar configuración mediante variables de entorno y realizar un despliegue funcional para la demostración.
6.- Presentar la solución en vivo y responder preguntas técnicas sobre análisis, arquitectura, implementación, pruebas y despliegue.
Un proyecto sin trazabilidad entre el análisis, la arquitectura, el código y las pruebas se considerará incompleto.
La revisión del proyecto final será realizada por el maestro mediante presentación y demostración funcional.

Se recomienda desarrollar e integrar el proyecto con anticipación, realizando commits frecuentes y verificando continuamente el pipeline de CI/CD.

Problema propuesto

Desarrollo e implementación del sistema correspondiente a la empresa asignada en las Unidades 2 y 3.

Antecedentes:
Durante las Unidades 2 y 3 cada equipo trabajó sobre un caso de negocio específico y desarrolló artefactos de análisis y diseño. El PIA representa la integración y culminación de ese trabajo: los alumnos deberán convertir los requerimientos, reglas, procesos y decisiones previamente definidos en una solución de software funcional, verificable y desplegable.

Solicitud:
Según la empresa que les fue asignada, el equipo deberá integrar, refinar e implementar una solución de software que satisfaga el dominio y los requerimientos previamente trabajados. El proyecto deberá incluir obligatoriamente:

• **ANÁLISIS**: BRD completo; mínimo 15 User Stories con criterios de aceptación en Gherkin; BPMN completo; glosario de dominio.

• **ARQUITECTURA**: diagramas DDD y UML; patrones justificados; aplicación de principios SOLID; ADRs y documento de decisiones técnicas.

• **IMPLEMENTACIÓN:** código funcional con los componentes del curso; modelo DDD; Specification; State Machine; API REST.

• **CALIDAD Y DEFENSA**: suite de pruebas con cobertura mínima de 80%; pruebas unitarias y de integración; pipeline CI/CD activo; demostración en vivo; respuestas técnicas.

• **PRODUCCIÓN:** Docker Compose para DB + App + Tests en un entorno reproducible; variables de entorno para separar configuración del código (12-Factor App); despliegue básico en Railway, Render, Fly.io o plataforma equivalente; Swagger UI/OpenAPI activo en producción para la demostración.

Trazabilidad obligatoria:
Los elementos del análisis deben corresponder con la implementación. Las reglas y criterios definidos en BRD/User Stories/Gherkin/BPMN deberán poder identificarse en el modelo de dominio, casos de uso, código y pruebas.

Estructura mínima esperada del repositorio GitHub:

```
mi-proyecto-cln/
├── docs/
│   ├── BRD.md
│   ├── architecture/        (ADRs, diagramas y decisiones técnicas)
│   └── api/                 (especificación OpenAPI)
├── src/
│   ├── domain/              (entidades, Value Objects, agregados, eventos)
│   ├── application/         (servicios y casos de uso)
│   ├── infrastructure/      (repositorios, DB, APIs externas)
│   └── api/                 (controllers/endpoints REST)
├── tests/                   (unitarias + integración)
├── .github/workflows/       (CI/CD)
├── docker-compose.yml
├── .env.example             (sin secretos reales)
└── README.md                (instrucciones técnicas, ejecución y demo)
```

El repositorio deberá contener historial de commits del equipo. No se aceptará como evidencia suficiente un programa aislado sin documentación, pruebas, arquitectura y automatización asociadas.

Producto Integrador de Aprendizaje, Criterios de evaluación

1.- ANÁLISIS DEL NEGOCIO (25% del PIA = 6.25 puntos)		Sí	En parte	No
1	BRD completo, coherente con la empresa asignada y refinado a partir del trabajo de U2/U3.	**1.50**	**0.75**	**-3.13**
2	≥15 User Stories con criterios de aceptación Gherkin + BPMN completo + glosario de dominio, con trazabilidad entre artefactos.	**4.75**	**2.25**	**-3.12**

2.- ARQUITECTURA (25% del PIA = 6.25 puntos)		Sí	En parte	No
1	Modelo/diagramas DDD: entidades, Value Objects, agregados, servicios/eventos de dominio según aplique.	1.50	50%	-1.50
2	Diagramas UML/arquitectura correctos y consistentes con la solución.	1.25	50%	-1.25
3	Patrones de diseño seleccionados y justificados de acuerdo con el problema.	1.25	50%	-1.25
4	Aplicación demostrable de principios SOLID.	1.00	**50%**	**-1.00**
5	ADRs y documento de decisiones técnicas: contexto, decisión, alternativas y consecuencias.	1.25	50%	-1.25
6	Repositorio GitHub organizado conforme a la estructura mínima indicada.	Requisito	**—**	**—**
7	docs/ con BRD, architecture/ y api/OpenAPI.	Requisito	—	—
8	src/ separado en domain, application, infrastructure y api.	Requisito	—	—
9	README técnico con instrucciones de instalación, ejecución, pruebas y demostración.	Requisito	—	—
10	.env.example sin credenciales ni secretos reales.	Requisito	—	—
11	Coherencia y trazabilidad entre análisis, arquitectura e implementación.	Requisito	—	—
12	Historial de commits que evidencie el desarrollo del proyecto.	Requisito	—	—
13	La arquitectura debe corresponder al caso de negocio asignado; no se aceptarán diagramas genéricos sin relación con el código.	Obligatorio	—	—

3.- IMPLEMENTACIÓN (25% del PIA = 6.25 puntos)		Puntos
1	Código funcional que implemente DDD + Specification + State Machine + API REST, coherente con el dominio y las reglas de la empresa asignada.	4.25
2	Persistencia/DB, documentación OpenAPI/Swagger y funcionamiento integral de los casos de uso principales.	2.00
3	**La solución no ejecuta, no implementa los componentes obligatorios o no corresponde al análisis presentado.**	**-100% del proyecto**

4.- CALIDAD + DEFENSA (25% del PIA = 6.25 puntos)		
1.- Pruebas unitarias y de integración con cobertura ≥80%.	2.- Pipeline CI/CD activo en .github/workflows/.	3.- Docker Compose reproducible para DB + App + Tests.
4.- Variables de entorno + despliegue funcional + Swagger UI/OpenAPI activo.	5.- Demo en vivo del sistema y sus principales reglas/estados.	6.- Defensa: respuestas técnicas sobre análisis, arquitectura, código, pruebas y despliegue.

Archivos entregables	
Antes de agendar la presentación y defensa, el equipo deberá subir en la sección de PIA los entregables y proporcionar acceso al repositorio GitHub. El repositorio deberá corresponder a la versión presentada.	
Reporte final PDF (BRD + arquitectura + evidencias + conclusiones)	**Repositorio GitHub (código + docs + tests + CI/CD + Docker + README)**
Entregar además la presentación de apoyo para la defensa y el enlace/URL del despliegue funcional. Si la plataforma solicita un archivo comprimido, incluir los documentos de entrega en ZIP/RAR y colocar en el README la URL del repositorio y del sistema desplegado.	

Cuadro de avance, actividades recomendadas por semana para la realización del proyecto final					
1	2	3	4	5	6
Revisión e integración final de BRD, ≥15 User Stories/Gherkin, BPMN y glosario de U2/U3.	Definición/refinamiento de arquitectura: DDD, UML, patrones, SOLID y ADRs.	Implementación del dominio, casos de uso, Specification, State Machine, persistencia y API.	Pruebas unitarias/integración; alcanzar ≥80% coverage; configurar GitHub Actions CI/CD.	Docker Compose, variables de entorno, despliegue y Swagger; documentación final y README.	Demostración en vivo y defensa técnica del proyecto.
Asesorías / revisión de avance				Presentación y defensa	